

The complete training loop

Your network grows from thirteen parameters to 1,057 and meets real data: 1,347 handwritten digits. Four ideas turn a single gradient step into a training run you can trust — epochs, mini-batches, shuffling and a learning rate chosen from evidence — and one picture tells you whether it worked.

Unit 3 · Libraries

80 minutes

Lab 11 · the loop on real digits

Still pure NumPy

The data

64–16–1

The loop

Learning rate

Lab 11

Before Tuesday

PART 1 · 12 MINUTES

Real data: 8×8 handwritten digits

Small enough for a laptop CPU, real enough to be proud of.

1

scikit-learn ships a dataset of 1,797 handwritten digits, each an 8×8 grid of grey levels from 0 to 16 — the same object as the grid you sliced in session 5. Flattened, each image is 64 numbers. We ask a question the session-14 network can answer without any new mathematics: **is this digit even?**

1,797

IMAGES

64

PIXELS EACH

1,347

FOR TRAINING

450

HELD BACK TO TEST

Two preparation steps, both with a reason. Divide the pixels by 16 so every input lies between 0 and 1 — session 19 shows what happens if you forget. And split before anything else, with a fixed seed, so the 450 test images are never seen during training.



Why not all ten digits?

Ten classes need a softmax output and a multi-class loss — one new idea too many for today. Even-versus-odd uses exactly the network, loss and backward pass you derived a week ago, so every line of today's code is something you have already understood.

PART 2 · 10 MINUTES

The same network, wider

2

	Session 13	Today
Inputs	2	64
Hidden neurons	3	16
Outputs	1	1
<code>W1</code> shape	(2, 3)	(64, 16)
Parameters	13	$64 \cdot 16 + 16 + 16 + 1 = \mathbf{1,057}$
Training examples	4	1,347

Not one line of `forward` or `backward` changes. That is the payoff of writing everything with matrices and shapes: the code does not know or care how wide the layers are. What does change is how we use the gradients — because computing them on all 1,347 images for every single step is no longer the obvious choice.

PART 3 · 25 MINUTES

Epochs, mini-batches, shuffling

Four nested ideas, and the order they go in.

3



Epoch

One complete pass through the training set. Forty epochs means every image is seen forty times.

Mini-batch

A small slice of examples — 32 today. Compute the gradient on the slice, take a step, move to the next slice.



Shuffle

A fresh random order at the start of every epoch, so batches differ each time and no pattern in the file's order leaks into training.



Record

After each epoch, measure the full training loss and the test accuracy. These two curves are how you know what happened.

the complete loop — Lab 11 asks you to write the middle

PYTHON

```
def train(lr, batch_size, epochs, seed=1):
    p = init_params()
    rng = np.random.default_rng(seed)
    losses, accs = [], []
    for epoch in range(epochs):
        idx = rng.permutation(len(X_train))           # shuffle row numbers
        for start in range(0, len(X_train), batch_size):
            b = idx[start:start + batch_size]         # one mini-batch
            g = backward(p, X_train[b], y_train[b])   # session 14
            for k in p:
                p[k] -= lr * g[k]                     # session 9
            losses.append(bce(p, X_train, y_train))
            accs.append(accuracy(p, X_test, y_test))
    return p, losses, accs
```

WHY MINI-BATCHES WIN

A gradient from 32 random images is a noisy estimate of the gradient from all 1,347 — but it costs a fortieth as much. **Forty-two noisy steps per epoch beat one precise step**, and the noise even helps a little, jostling the walk out of shallow dips. With 1,347 images and batches of 32 there are 43 updates per epoch.



Shuffle the row numbers, never the arrays separately

Index `x_train` and `y_train` with the *same* slice of `idx`. Shuffling one without the other still runs, still produces a loss curve — and trains the network on images paired with the wrong labels. It is one of the four bugs in Lab 13.

PART 4 · 18 MINUTES

Choosing the learning rate from evidence

Four runs, forty epochs each, the same seed.

4

Learning rate	Final training loss	Test accuracy	What the curve looked like
0.01	0.655	0.767	barely moving — still near the 0.693 of a coin toss
0.1	0.207	0.902	steady, but still falling when we stopped
1.0	0.040	0.976	fast drop, then a smooth flattening
5.0	0.110	0.933	falls, but jagged — steps overshoot and recover

Read the table the way session 9 taught you to read η . Too small is not wrong, only slow; too large is noisy and eventually diverges. The right choice is the largest rate whose curve is still smooth. And judge from the **curves**, not the last number: a run that ended well by luck on a jagged curve will not do so twice.

A NUMBER TO BE PROUD OF, HONESTLY

Guessing the more common class gets **46.7%** of the test images right. Your network gets **97.6%** — on images it has never seen, with a model you built from matrix multiplications and the chain rule. Always report the baseline beside the result.

Three questions

Answer before Lab 11.

Q1. How many parameter updates happen in 40 epochs with batch size 32? 43 batches per epoch (the last one has 3 images) $\times 40 = 1,720$ updates.

Q2. The training loss after epoch 1 is higher than the starting loss of 0.69. Broken? Not necessarily. With $\eta = 1.0$ the first steps overshoot while the weights are tiny and uncoordinated. Look at the trend over epochs, not one point.

Q3. Why record test accuracy every epoch if we must not use the test set to make decisions? Today we only watch it, to learn what a training run looks like. Choosing the learning rate from it would contaminate it — session 22 introduces a separate validation set for exactly that job.

LAB 11 · 22 MINUTES IN CLASS

The training loop on real digits

5

Lab 11 — `CSC4315_Lab11_Training_Loop_On_Digits`. Data loading, the network and plotting are handed out; the loop is yours. The assert asks for at least 95% test accuracy.

Before Tuesday

1. **Finish Lab 11** and keep its final accuracy — Tuesday you try to beat it in four lines.
2. **Count** the lines in your full Lab 11 notebook that do real work: data, network, backward, loop, evaluation.



Session 17 — Tuesday 03 November, 16:00

The same model in scikit-learn. Your file becomes four lines — and every line that disappears gets named.